Relatório Técnico - Fase 2: Tech Challenge

Projeto: Food Backend 2 - Sistema de Gestão para Restaurantes

Autor: Thiago Henrique Alves Ferreira / RM369442

1. Introdução

Este documento apresenta a arquitetura, as decisões técnicas e as instruções de execução do projeto **Food Backend 2**, desenvolvido como requisito para a Fase 2 do Tech Challenge da Pós Tech em Arquitetura e Desenvolvimento Java. O sistema consiste em uma plataforma de backend para o gerenciamento de restaurantes, usuários e cardápios, construída com foco em clean code, alta coesão, baixo acoplamento e testabilidade.

2. Padrão Arquitetural

Para garantir que as regras de negócio fossem o coração da aplicação, todo o projeto foi estruturado seguindo os princípios da **Clean Architecture** (Arquitetura Limpa). A divisão foi feita baseada no conceito de *Package by Feature* (Pacotes por Funcionalidade), isolando o domínio das tecnologias de infraestrutura.

- **Camada Core:** Contém as entidades de negócio puras (Usuário, Restaurante, ItemCardapio), Casos de Uso (UseCases) e as interfaces (Gateways). Esta camada não possui dependências de frameworks externos como Spring ou JPA.
- **Camada de Infraestrutura:** Contém os controladores REST, os repositórios Spring Data JPA, as configurações de Injeção de Dependência e os adaptadores de banco de dados.

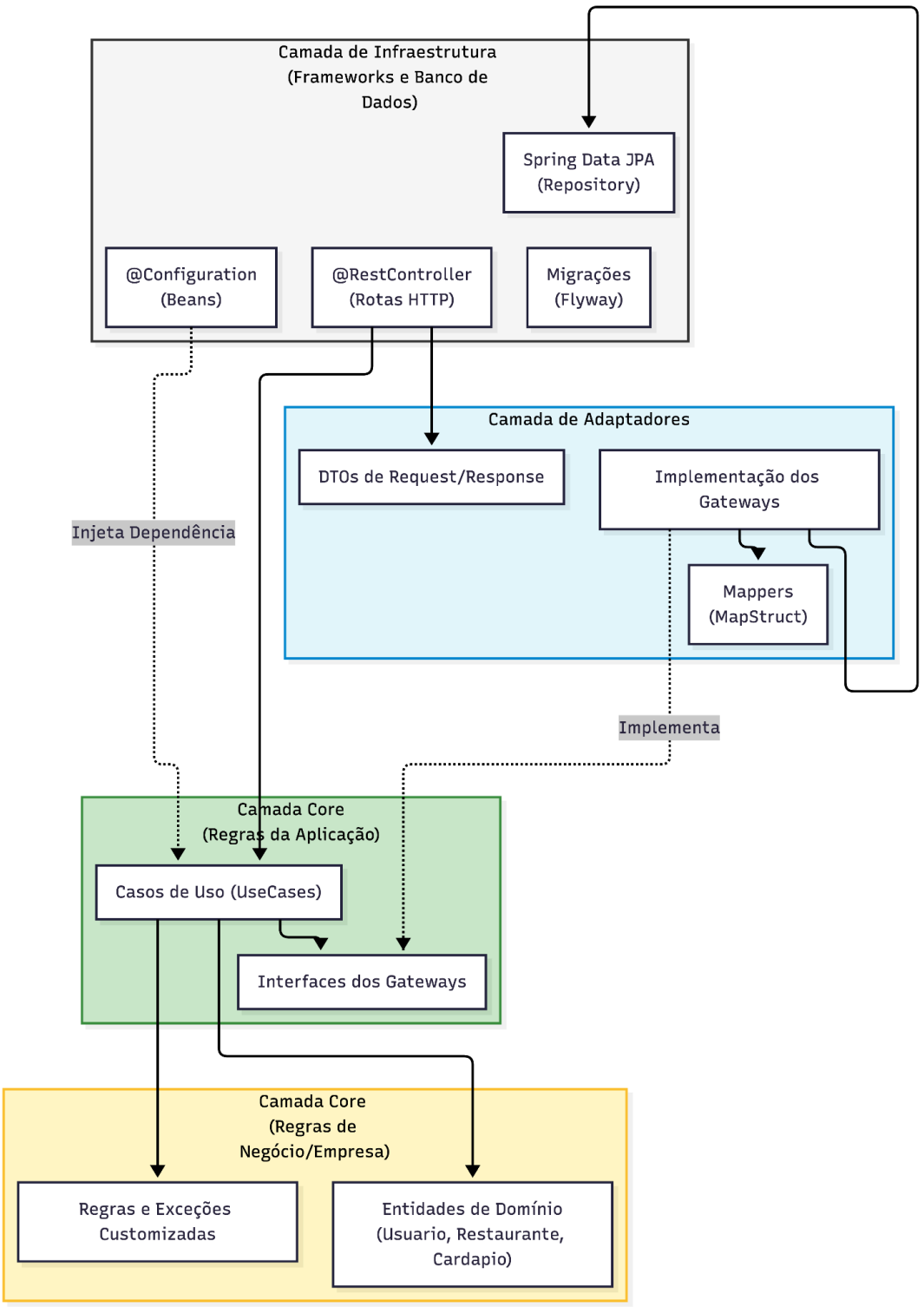

Figura 1: Visão geral do isolamento da camada Core em relação aos adaptadores e frameworks de infraestrutura.

3. Modelagem de Dados e Entidades

O sistema gerencia o fluxo de criação de contas e o catálogo de produtos. Para otimização de infraestrutura, os itens do cardápio não armazenam a imagem física no banco de dados, mas sim o seu caminho de rede (`fotoPath`). A persistência é gerenciada pelo Spring Data JPA, e a evolução do esquema do banco de dados é versionada de forma automatizada via **Flyway**.

As principais entidades e seus relacionamentos são:

- **TipoUsuario e Usuario:** Classificação de acesso no sistema (ex: Cliente ou Dono de Restaurante).
- **Restaurante:** Estabelecimento atrelado a um Usuário dono.
- **ItemCardapio:** Produtos atrelados a um Restaurante específico.

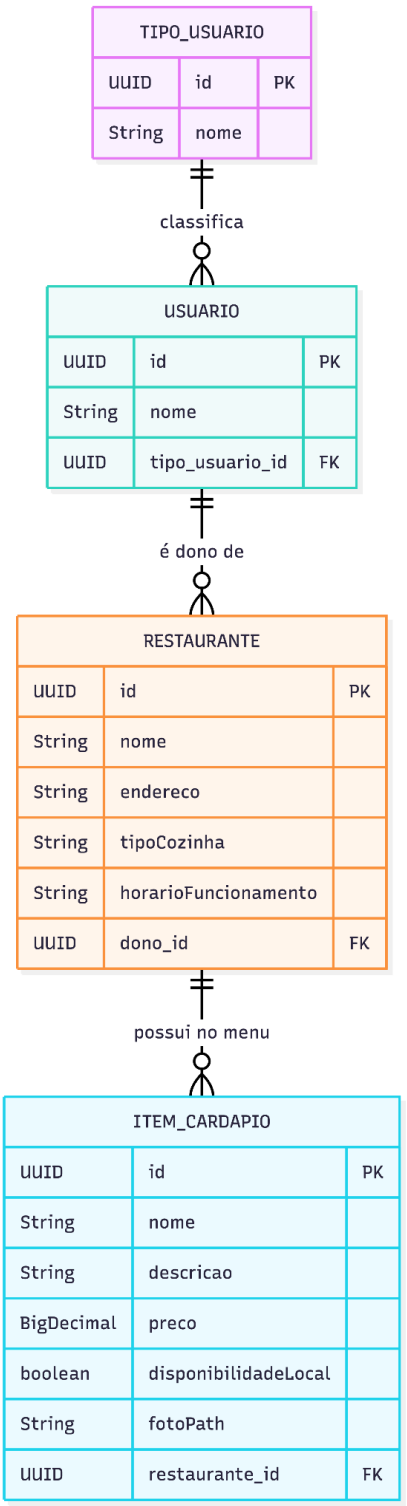

Figura 2: Modelagem lógica e relacional das tabelas geradas no banco de dados MySQL.

4. Infraestrutura e Orquestração (Docker)

Para facilitar o processo de deploy e testes locais do avaliador, o ambiente foi totalmente containerizado.

Foi utilizado um **Dockerfile com Multi-stage Build** para compilar a aplicação Java 21 usando o Maven e, em seguida, gerar uma imagem leve apenas com a JRE para execução. O ambiente completo é orquestrado pelo **Docker Compose**, que sobe o banco de dados MySQL e a API simultaneamente, garantindo a inicialização correta via *healthchecks*.

Passo a passo para execução:

1. Na raiz do projeto, execute: `docker compose up -d --build`
2. A API ficará disponível na porta `8080`.
3. O banco de dados iniciará na porta `3306` e as migrações do Flyway rodarão automaticamente.

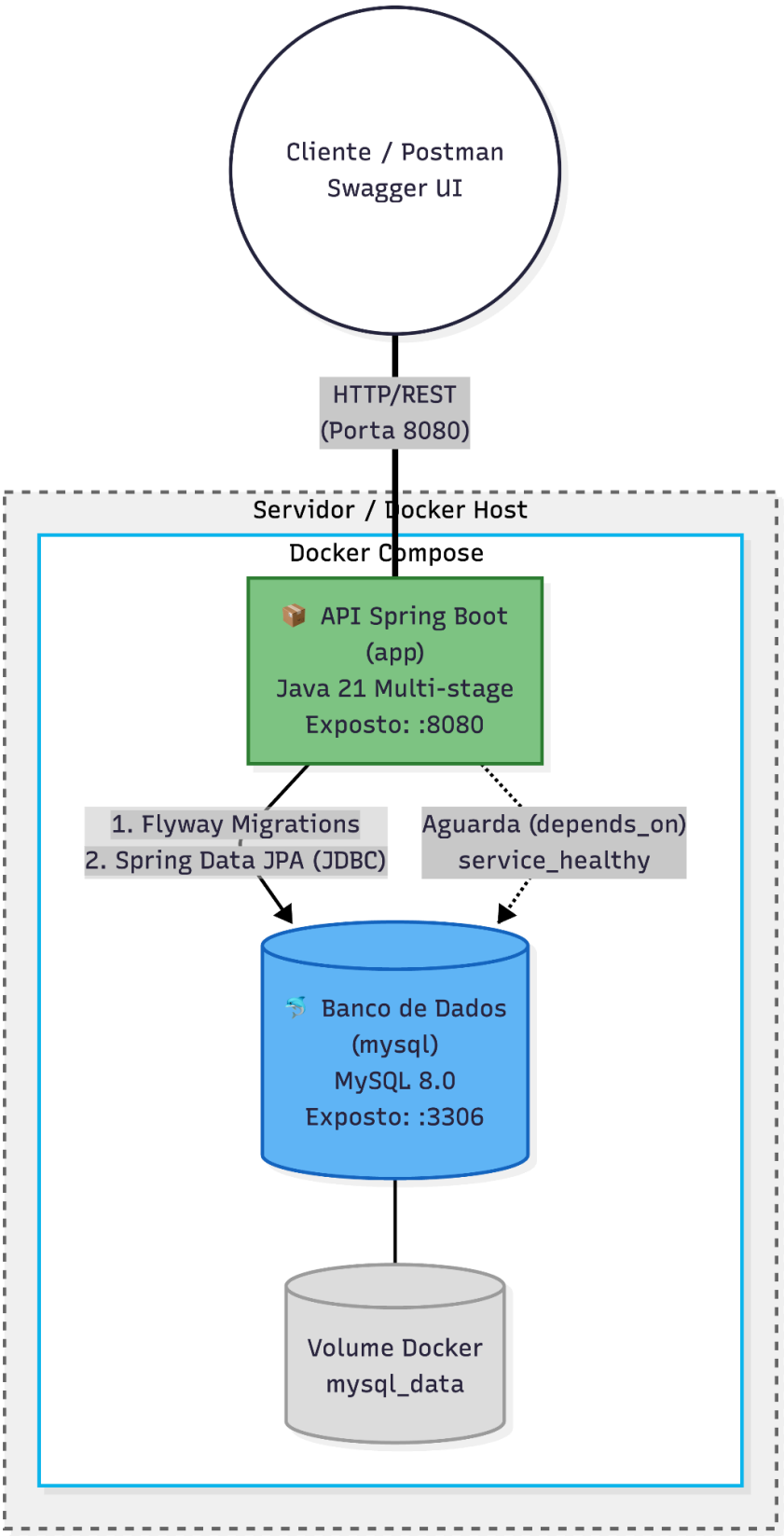

Figura 3: Topologia de infraestrutura orquestrada via Docker Compose, destacando portas expostas e volumes.

5. Qualidade de Código e Testes

O projeto possui uma rigorosa esteira de validação para garantir a integridade da Clean Architecture:

- **Testes Unitários:** O núcleo do sistema (camada Core) possui **100% de cobertura de código** aferida pelo plugin **JaCoCo**, superando a meta estabelecida de 80%.
- **Testes de Integração:** Foram implementados testes de ponta a ponta (E2E) utilizando **RestAssured** e **Testcontainers**. Durante a fase de testes, o Spring Boot orquestra a subida de um banco MySQL descartável no Docker, validando o fluxo completo (HTTP -> Controller -> UseCase -> BD -> Resposta HTTP).

food-backend2-clean-arch

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.thiagoferreira.foodbackend2cleanarch.usuario.core.usecase		100%		100%	0	25	0	68	0	20	0	10
com.thiagoferreira.foodbackend2cleanarch.usuario.infra.controller		100%		n/a	0	12	0	49	0	12	0	2
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.usecase		100%		100%	0	13	0	38	0	10	0	5
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.controller		100%		n/a	0	6	0	40	0	6	0	1
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.usecase		100%		100%	0	13	0	37	0	10	0	5
com.thiagoferreira.foodbackend2cleanarch.usuario.infra.persistence.mapper		100%		100%	0	10	0	38	0	6	0	2
com.thiagoferreira.foodbackend2cleanarch.usuario.infra.gateway		100%		n/a	0	9	0	22	0	9	0	2
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.persistence.mapper		100%		100%	0	10	0	35	0	4	0	2
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.controller		100%		n/a	0	6	0	33	0	6	0	1
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.persistence.mapper		100%		100%	0	5	0	33	0	3	0	1
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.domain		100%		n/a	0	12	0	31	0	12	0	1
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.gateway		100%		n/a	0	7	0	14	0	7	0	1
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.gateway		100%		n/a	0	6	0	13	0	6	0	1
com.thiagoferreira.foodbackend2cleanarch.usuario.core.exception		100%		n/a	0	25	0	27	0	25	0	5
com.thiagoferreira.foodbackend2cleanarch.usuario.core.domain		100%		n/a	0	11	0	27	0	11	0	2
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.domain		100%		n/a	0	9	0	23	0	9	0	1
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.exception		100%		n/a	0	20	0	21	0	20	0	4
com.thiagoferreira.foodbackend2cleanarch.usuario.core.mapper		100%		100%	0	7	0	14	0	4	0	2
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.rule		100%		100%	0	6	0	11	0	2	0	1
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.exception		100%		n/a	0	15	0	18	0	15	0	3
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.rule		100%		100%	0	6	0	11	0	2	0	1
com.thiagoferreira.foodbackend2cleanarch.usuario.core.rule		100%		100%	0	5	0	10	0	2	0	2
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.mapper		100%		100%	0	3	0	11	0	2	0	1
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.mapper		100%		100%	0	3	0	10	0	2	0	1
com.thiagoferreira.foodbackend2cleanarch.util.rule		100%		100%	0	4	0	5	0	2	0	1
com.thiagoferreira.foodbackend2cleanarch.util.exception		100%		n/a	0	5	0	7	0	5	0	2
Total	0 of 2.015	100%	0 of 82	100%	0	253	0	646	0	212	0	60

Figura 4: Cobertura de código visualizada no JaCoCo.

6. Documentação da API e Endpoints

A interface de comunicação segue os padrões RESTful, com payloads no formato JSON. O sistema conta com três níveis de documentação:

1. **Documentação de Código (JavaDoc):** Regras de negócio, UseCases e Gateways estão documentados em HTML na pasta `/docs` do repositório.
2. **Documentação Interativa (Swagger UI):** Acessível via `http://localhost:8080/swagger-ui.html` quando a aplicação está no ar.
3. **Collections de Teste:** O arquivo `food-backend2-postman.json` encontra-se na pasta `/docs/postman` do projeto contendo as variáveis de ambiente e requisições prontas para testes de criação, busca, atualização e exclusão de todas as entidades.

SEARCH Search

food-backend2-clean-arch 0.0.1-SNAPSHOT API

Packages

Package	Description
com.thiagoferreira.foodbackend2cleanarch	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.domain	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.dto	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.exception	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.gateway	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.mapper	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.rule	
com.thiagoferreira.foodbackend2cleanarch.cardapio.core.usecase	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.config	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.controller	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.dto	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.gateway	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.persistence.entity	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.persistence.mapper	
com.thiagoferreira.foodbackend2cleanarch.cardapio.infra.persistence.repository	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.domain	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.dto	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.exception	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.gateway	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.mapper	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.rule	
com.thiagoferreira.foodbackend2cleanarch.restaurante.core.usecase	
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.config	
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.controller	
com.thiagoferreira.foodbackend2cleanarch.restaurante.infra.dto	

Figura 5: Overview do JavaDoc.

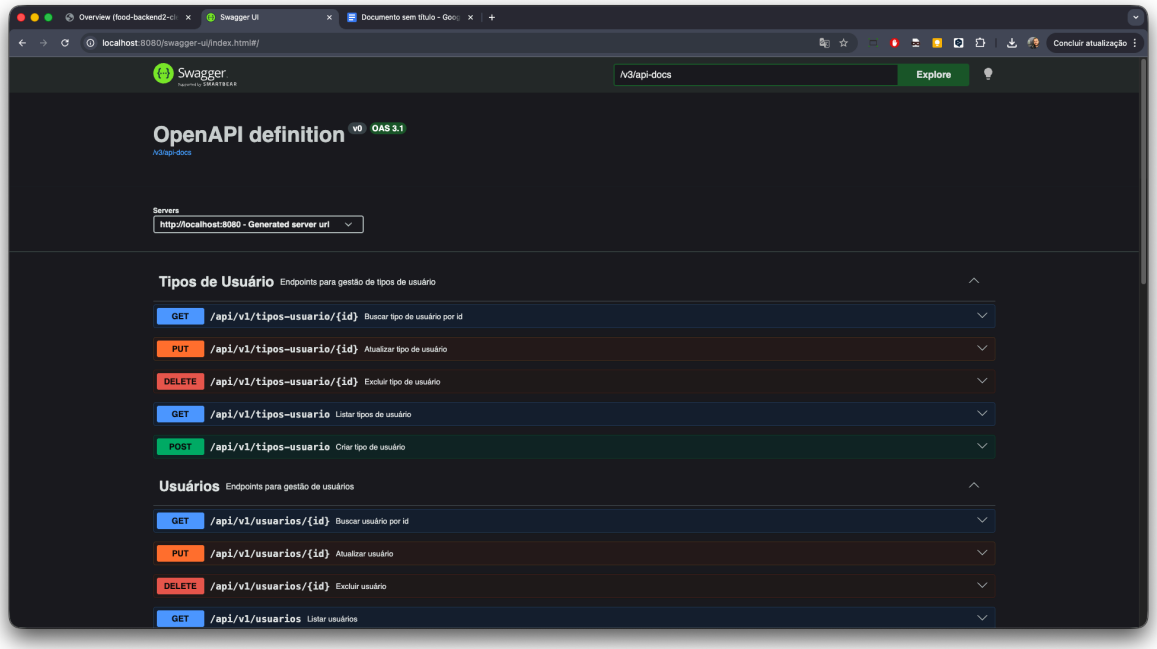

Figura 6: Visão da documentação Swagger.

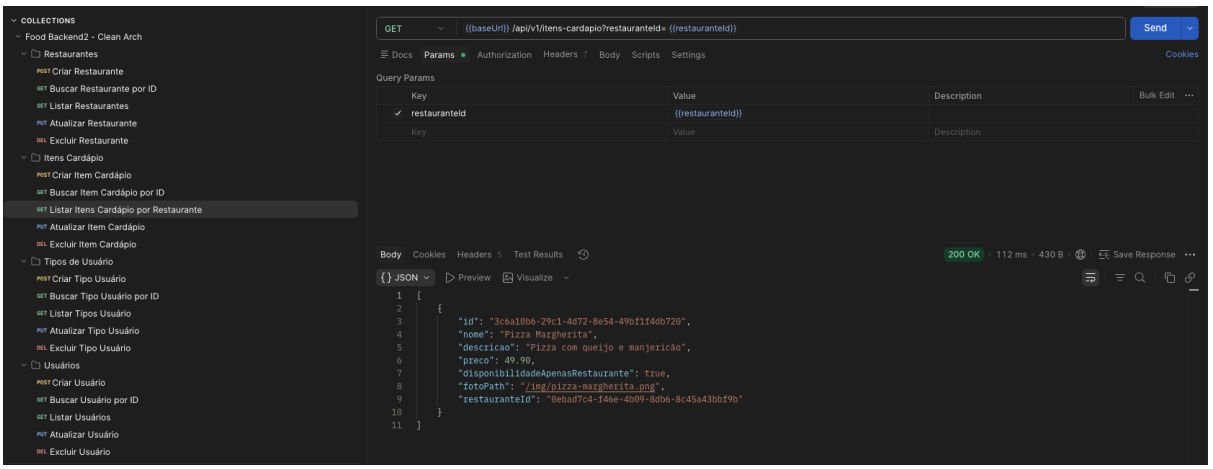

Figura 7: Visão da Collection FoodBackend2 - Clean Arch no Postman.

7. Conclusão

A implementação cumpriu e excedeu todos os requisitos funcionais e não-funcionais estipulados para a Fase 2. A separação clara de responsabilidades confere ao projeto um alto nível de manutenibilidade, preparando a base de código estruturalmente para as futuras integrações (como por exemplo o módulo de Pedidos).